

**LAPORAN PRAKTIKUM**  
**PEMROGRAMAN BERORIENTASI OBJEK**  
**MODUL 11**  
**ERROR HANDLING**



**Disusun Oleh:**

**Sisca Monica Nasution 105224042**

**PROGRAM STUDI ILMU KOMPUTER**  
**FAKULTAS SAINS DAN KOMPUTER**  
**UNIVERSITAS PERTAMINA**  
**2026**

## **I. Pendahuluan**

Dalam pengembangan aplikasi berbasis Java, program yang berjalan di dunia nyata tidak selalu menerima input yang sempurna dari pengguna. Pengguna bisa saja salah mengetik huruf di kolom yang seharusnya diisi angka, memasukkan data yang tidak sesuai format, atau memesan sesuatu yang sudah tidak tersedia. Jika kondisi-kondisi ini tidak ditangani dengan benar, program akan berhenti secara tiba-tiba atau crash, yang tentu saja sangat merugikan pengguna.

Untuk mengatasi hal tersebut, Java menyediakan mekanisme Exception Handling, yaitu cara terstruktur untuk menangkap dan menangani kesalahan yang terjadi saat program dijalankan (runtime error). Mekanisme ini menggunakan blok try-catch-finally, serta keyword throw dan throws untuk melempar dan mendelegasikan penanganan error.

Pada praktikum ini, konsep tersebut diterapkan dalam sebuah proyek bernama "JAVA EXPRESS", prototipe sistem reservasi tiket kereta api berbasis CLI (Command Line Interface). Program ini dirancang agar tetap stabil dan tidak crash meski pengguna memberikan input yang salah. Sistem ini memiliki tiga fitur utama: melihat jadwal kereta, memesan tiket, dan keluar dari aplikasi.

Program dibangun dari enam file Java, yaitu Kereta.java, SistemReservasi.java, Main.java, dan tiga class custom exception: DataPenumpangTidakValidException, RuteTidakDitemukanException, dan TiketHabisException. Masing-masing exception dirancang untuk menangani jenis kesalahan yang berbeda sesuai konteks bisnis aplikasi.

## II. Variabel

### Class Kerets

| No | Nama Variabel     | Tipe data | Fungsi                                                   |
|----|-------------------|-----------|----------------------------------------------------------|
| 1  | <i>KodeKereta</i> | String    | Menyimpan kode unik pengenalan kereta, contohnya “K01”   |
| 2  | namaKereta        | String    | Menyimpan nama kereta                                    |
| 3  | rute              | String    | Menyimpan rute perjalanan kereta                         |
| 4  | sisakursi         | int       | Menyimpan jumlah kursi yang masih tersedia untuk dipesan |

### Class SisterReservasi

| No | Nama Variabel | Tipe data    | Fungsi                                                                           |
|----|---------------|--------------|----------------------------------------------------------------------------------|
| 1  | daftarKereta  | List<kereta> | Menyimpan semua objek kereta yang tersedia dalam sistem sebagai daftar berurutan |

### Class TiketHabisException

| No | Nama Variabel | Tipe data | Fungsi                                                                                        |
|----|---------------|-----------|-----------------------------------------------------------------------------------------------|
| 1  | namaKereta    | String    | Menyimpan nama kereta yang kursinya tidak mencukupi, agar bisa ditampilkan saat error terjadi |
| 2  | sisakursi     | int       | Menyimpan sisa kursi aktual saat error terjadi, sebagai informasi tambahan kepada pengguna    |

### Class Main

| No | Nama Variabel | Tipe data       | Fungsi                                                |
|----|---------------|-----------------|-------------------------------------------------------|
| 1  | sistem        | SistemReservasi | Objek utama yang menjalankan seluruh logika reservasi |
| 2  | scanner       | Scanner         | Alat pembaca input dari keyboard pengguna             |

|   |          |         |                                                                                   |
|---|----------|---------|-----------------------------------------------------------------------------------|
| 3 | berjalan | boolean | Penanda apakah program masih harus terus berjalan; jika false, loop menu berhenti |
| 4 | pilihan  | int     | Menyimpan angka menu yang dipilih pengguna                                        |

### III. Constructor dan Method

| No | Nama Metode                             | Jenis Metode       | Fungsi                                                                                                                                       |
|----|-----------------------------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Kereta(String, String, String, int)     | <i>Constructor</i> | Membuat objek kereta baru dengan mengisi semua atributnya sekaligus kode, nama, rute dan sisa kursi awal                                     |
| 2  | getKodeKereta()                         | Functional         | Mengembalikan nilai kode kereta                                                                                                              |
| 3  | getNamaKereta()                         | Functional         | Mengembalikan nilai nama kereta                                                                                                              |
| 4  | getRute()                               | Functional         | Mengembalikan nilai rute kereta                                                                                                              |
| 5  | getSisaKursi()                          | Functional         | Mengembalikan jumlah sisa kursi yang masih tersedia                                                                                          |
| 6  | kurangKursi(int)                        | procedural         | Mengurangi sisaKursi sesuai jumlah tiket yang berhasil                                                                                       |
| 7  | SistemReservasi                         | Constructor        | Menginisialisasi daftar kereta dan otomatis mengisi dua data awal: Argo Bromo (K01) dan Parahyangan (K02)                                    |
| 8  | getDaftarKereta()                       | Functional         | Mengembalikan seluruh daftar kereta yang tersimpan, digunakan untuk menampilkan jadwal                                                       |
| 9  | pesanTiket(String, String, String, int) | Procedural         | Memproses pemesanan tiket dengan validasi berlapis: validasi NIK, validasi jumlah tiket, pencarian kereta, dan pengecekan ketersediaan kursi |

|    |                                          |             |                                                                                                  |
|----|------------------------------------------|-------------|--------------------------------------------------------------------------------------------------|
| 10 | DataPenumpangTidakValidException(String) | constructor | Membuat exception dengan pesan error saat NIK penumpang tidak valid                              |
| 11 | RuteTidakDitemukanException(String)      | constructor | Membuat exception dengan pesan error saat kode kereta tidak ditemukan di sistem                  |
| 12 | TiketHabisException(String, String, int) | constructor | Membuat exception yang membawa pesan error sekaligus informasi nama kereta dan sisa kursi aktual |
| 13 | getNamaKereta()                          | Functional  | Getter di class TiketHabisException untuk mengambil nama kereta yang kursinya tidak cukup        |
| 14 | getSisaKursi()                           | Functional  | Getter di class TiketHabisException untuk mengambil sisa kursi saat error terjadi                |

#### IV. Dokumentasi dan Pembahasan Code

##### A. Class Kereta

Class Kereta berfungsi sebagai cetak biru (*blueprint*) yang merepresentasikan satu kereta dalam sistem. Seluruh atributnya dideklarasikan sebagai private sesuai prinsip *encapsulation*, sehingga data hanya bisa diakses melalui method getter yang disediakan. Terdapat satu method khusus bernama kurangiKursi() yang bertugas mengurangi sisaKursi ketika pemesanan berhasil dilakukan. Method ini tidak mengembalikan nilai (*void*) karena tugasnya hanya mengubah state objek, bukan menghitung sesuatu.

```
public void kurangiKursi(int jumlah) {
    this.sisaKursi -= jumlah;
}
```

##### B. Tiga Custom Exception

Bagian ini adalah inti dari penerapan materi Error Handling. Program membuat tiga class exception buatan sendiri, masing-masing untuk jenis kesalahan yang berbeda.

DataPenumpangTidakValidException merupakan Unchecked Exception karena meng-*extend* RuntimeException. Artinya, method yang melempar exception ini tidak wajib mendeklarasikannya dengan throws di tanda tangannya. Exception ini dipilih sebagai *unchecked*

karena kesalahan NIK dianggap sebagai kesalahan pemrograman atau input pengguna yang seharusnya bisa dicegah sebelumnya.

```
public class DataPenumpangTidakValidException extends RuntimeException {  
  
    public DataPenumpangTidakValidException(String pesan) {  
  
        super(pesan);  
  
    }  
  
}
```

RuteTidakDitemukanException merupakan Checked Exception karena meng-*extend* Exception. Exception jenis ini wajib ditangani oleh pemanggil method, baik dengan try-catch maupun dengan mendeklarasikan throws di signature method.

TiketHabisException juga merupakan Checked Exception, namun dengan tambahan dua atribut khusus: namaKereta dan sisaKursi. Ini adalah contoh *custom exception* yang kaya informasi, selain pesan error, exception ini juga membawa data kontekstual yang sangat berguna untuk ditampilkan kepada pengguna.

```
public class TiketHabisException extends Exception {  
    private String namaKereta;  
    private int sisaKursi;
```

### C. Class SistemReservasi

Class ini adalah pusat kendali logika bisnis. Constructor-nya langsung mengisi dua data kereta awal ke dalam ArrayList saat objek dibuat, sehingga sistem sudah siap pakai tanpa perlu input manual.

Method terpenting adalah pesanTiket(), yang menjalankan empat validasi secara berurutan. Urutan validasi ini penting dan disengaja:

Pertama, NIK divalidasi menggunakan dua kondisi sekaligus: panjangnya harus tepat 16 karakter, dan seluruh karakternya harus berupa digit angka menggunakan matches("\\d+"). Jika gagal, langsung dilempar DataPenumpangTidakValidException.

```
if (nik.length() != 16 || !nik.matches("\\d+")) {  
    throw new DataPenumpangTidakValidException("NIK harus berisi  
16 digit angka!");  
}
```

Kedua, jumlah tiket divalidasi agar tidak nol atau negatif. Jika tidak valid, dilempar `IllegalArgumentException` yang merupakan exception bawaan Java.

Ketiga, sistem mencari kereta berdasarkan kode menggunakan perulangan `for`. Jika tidak ditemukan setelah seluruh daftar diperiksa, dilempar `RuteTidakDitemukanException`.

Keempat, sisa kursi dibandingkan dengan jumlah tiket yang diminta. Jika tidak mencukupi, dilempar `TiketHabisException` lengkap dengan informasi nama kereta dan sisa kursi aktual. Jika semua validasi lolos, `kurangiKursi()` dipanggil dan konfirmasi pemesanan ditampilkan.

Method ini mendeklarasikan `throws RuteTidakDitemukanException, TiketHabisException` karena keduanya adalah Checked Exception. `DataPenumpangTidakValidException` tidak perlu dideklarasikan karena sifatnya unchecked.

#### **D. Class Main**

Class Main adalah titik masuk program dan bertanggung jawab atas seluruh interaksi dengan pengguna. Strukturnya menggunakan dua lapis try-catch yang bekerja bersama.

Lapisan luar (try terluar) membungkus seluruh loop program dan memiliki blok finally. Blok finally ini menjamin bahwa `scanner.close()` dan pesan perpisahan kepada pengguna selalu dieksekusi tanpa terkecuali, bahkan jika terjadi error yang tidak tertangkap sekalipun.

```
try {  
  
    while (berjalan) {  
  
        System.out.println("\n=== SISTEM RESERVASI JAVA EXPRESS  
===");  
  
        System.out.println("1. Lihat Jadwal & Sisa Kursi");  
  
        System.out.println("2. Pesan Tiket");  
  
        System.out.println("3. Keluar");  
  
        System.out.print("Pilih menu: ");  
  
        try {  
  
            int pilihan = scanner.nextInt();
```

```
scanner.nextLine();

if (pilihan == 1) {

    System.out.println("\n--- JADWAL KERETA API ---");

    for (Kereta k : sistem.getDaftarKereta()) {

        System.out.println("Kode: " +
k.getKodeKereta() +
                                " | Nama: " +
k.getNamaKereta() +
                                " | Rute: " + k.getRute() +
                                " | Sisa Kursi: " +
k.getSisaKursi());

    }

} else if (pilihan == 2) {

    System.out.print("Masukkan Kode Kereta: ");

    String kode = scanner.nextLine();

    System.out.print("Masukkan NIK: ");

    String nik = scanner.nextLine();

    System.out.print("Masukkan Nama Penumpang: ");

    String nama = scanner.nextLine();

    System.out.print("Masukkan Jumlah Tiket: ");

    int jumlah = scanner.nextInt();

    scanner.nextLine();

}
```

```

        sistem.pesanTiket(kode, nik, nama, jumlah);

    } else if (pilihan == 3) {

        berjalan = false;

    } else {

        System.out.println("Pilihan menu tidak valid!");

    }

} catch (InputMismatchException e) {

    System.out.println("Error: Input harus berupa
angka!");

    scanner.nextLine();

} catch (DataPenumpangTidakValidException e) {

    System.out.println("Error Validasi Penumpang: " +
e.getMessage());

} catch (RuteTidakDitemukanException e) {

    System.out.println("Error Rute: " + e.getMessage());

} catch (TiketHabisException e) {

    System.out.println("Error Ketersediaan: " +
e.getMessage());

    System.out.println("Kereta: " + e.getNamaKereta() + "
| Sisa Kursi: " + e.getSisaKursi());

} catch (IllegalArgumentException e) {

    System.out.println("Error Input: " + e.getMessage());

}

}

} finally {

```

```

        scanner.close();

        System.out.println("\nTerima kasih telah menggunakan layanan
JAVA EXPRESS!");
    }

}

```

Lapisan dalam (try di dalam loop) menangkap semua exception yang mungkin terjadi dalam satu siklus menu. Terdapat lima blok catch yang ditangani:

- `InputMismatchException` : ditangkap saat pengguna mengetik huruf di kolom yang meminta angka (misalnya saat memilih menu). Setelah ditangkap, `scanner.nextLine()` dipanggil untuk membersihkan buffer input agar program tidak terjebak dalam *infinite loop*.
- `DataPenumpangTidakValidException` : menampilkan pesan bahwa format NIK salah.
- `RuteTidakDitemukanException` : menampilkan pesan bahwa kode kereta tidak ditemukan.
- `TiketHabisException` : menampilkan pesan beserta informasi nama kereta dan sisa kursi yang diambil lewat getter khusus milik exception ini.
- `IllegalArgumentException` : menangkap error saat jumlah tiket yang dimasukkan tidak valid (nol atau negatif)

## **V. Kesimpulan**

Dari praktikum ini dapat disimpulkan bahwa mekanisme Exception Handling dalam Java sangat penting untuk membangun program yang stabil dan tidak mudah crash. Penggunaan blok try-catch memungkinkan program menangkap dan merespons error dengan pesan yang informatif tanpa menghentikan jalannya aplikasi. Pemilihan jenis exception juga perlu disesuaikan dengan konteks, Checked Exception seperti `RuteTidakDitemukanException` dan `TiketHabisException` memaksa programmer menangani error sejak awal, sementara Unchecked Exception seperti `DataPenumpangTidakValidException` lebih fleksibel untuk error akibat input pengguna. Membuat custom exception terbukti membuat kode lebih mudah dibaca dan lebih ekspresif, terutama ketika exception tersebut juga membawa informasi tambahan yang relevan. Selain itu, blok finally menjamin bahwa resource seperti Scanner selalu ditutup dengan benar, dan urutan penulisan blok catch dari yang paling spesifik ke umum harus selalu diperhatikan agar tidak ada kode yang tidak terjangkau (unreachable code).

## **VI. Daftar Pustaka**